

Transfer Learning (bilimni ko'chirish) va oldindan o'qitilgan modellar (BERT, T5)

Tabiiy tilni qayta ishlash — 13-ma'ruza

A.A. Abdulali

11:30–12:50 · mavzu №13

7-iyul 2026

Prezentatsiya rejasi

- 1 Kirish: noldan o'qitmaymiz, oldindan o'qitilganni olamiz
- 2 Transfer learning: pre-training va fine-tuning
- 3 BERT: ikki tomonlama enkoder
- 4 T5: hamma narsa matndan matnga
- 5 Hugging Face ekotizimi
- 6 Xulosa va amaliyotga ko'prik

Takrorlash: o'tgan darsdan ikki savol

1-savol

L12 da Transformer arxitekturasini ko'rdik. Uni noldan o'qitish uchun qanday resurslar kerak?

2-savol

O'zbek tili uchun katta hajmli yorliqlangan korpus yetarli bo'lmasa, kuchli Transformer modelini qanday moslaymiz?

Takrorlash: o'tgan darsdan ikki savol

1-savol

L12 da Transformer arxitekturasini ko'rdik. Uni noldan o'qitish uchun qanday resurslar kerak?

1-javob

Juda katta korpus, ko'p GPU-soat va uzoq o'qitish jarayoni kerak. Har safar noldan boshlash amaliy jihatdan qimmat.

2-savol

O'zbek tili uchun katta hajmli yorliqlangan korpus yetarli bo'lmasa, kuchli Transformer modelini qanday moslaymiz?

2-javob

Oldindan o'qitilgan modelni olamiz va uni kichik datada **vazifaga moslaymiz** (*fine-tuning*). Bu — **bilimni ko'chirish** (*transfer learning*).

Dars yakunida nimalarni bajara olasiz?

- 1 Transfer learning (*bilimni ko'chirish*), pre-training (*oldindan o'qitish*) va fine-tuning (*vazifaga moslab sozlash*) bosqichlarini **farqlay olasiz**.
- 2 Oldindan o'qitilgan model resurslari cheklangan tillar uchun nega amaliy jihatdan muhimligini **tushuntira olasiz**.
- 3 Ikkilik klassifikatsiyada Binary Cross-Entropy (BCE, *ikkilik kross-entropiya*) yo'qotishini **hisoblay olasiz**: $\sigma(2.0) \approx 0.8808$, $BCE \approx 0.1269 \approx 0.127$ (o'quv misolida ≈ 0.128).
- 4 BERT (Bidirectional Encoder Representations from Transformers, *ikki tomonlama enkoder*) va T5 (Text-to-Text Transfer Transformer, *matndan matnga*) modellarining asosiy farqini **taqqoslay olasiz**.
- 5 Hugging Face ekotizimida fine-tuning oqimini: model, tokenizator, ma'lumotlar to'plami, o'qitish va baholash bosqichlarini **ko'rsata olasiz**.

Kerakli tayanch bilimlar

L12: Transformer va o'z-o'ziga e'tibor (*self-attention*); L2: logistik regressiya va sigmoid; L6/L9: softmax; matematika: exp, ln va ehtimol.

Bugungi dars rejasi

Bosqich	Mavzu	Darsdagi natija
1	Transfer learning va fine-tuning	BCE yo'qotishini sodda misolda qo'lda hisoblash
2	BERT: ikki tomonlama enkoder	niqoblangan tokenni kontekst asosida bashorat qilish
3	T5: matndan matnga yondashuv	vazifalarni matn→matn formatida ifodalash
4	Hugging Face ekotizimi	fine-tuning jarayonining amaliy oqimini ko'rsatish

Bu dars 13-amaliyotga olib keladi

Bugun oldindan o'qitilgan modelni kichik datada vazifaga moslash g'oyasini tushunamiz. $BCE \approx 0.128$ qiymati keyingi amaliyotda haqiqiy Trainer loss emas, balki BCE intuitsiyasini tekshiruvchi warm-up assert sifatida ishlatiladi.

Muammo: noldan o'qitish qimmat va sekin

Noldan o'qitish qayerda qiynaladi?

Katta korpus va hisoblash resursi

Kuchli Transformer modelini noldan o'qitish katta matn korpusi, ko'p GPU-soat va uzoq tajriba jarayonini talab qiladi.

Har vazifa uchun alohida xarajat

Sentiment, NER yoki tarjima kabi vazifalar uchun noldan boshlash amaliy jihatdan qimmat va sekin.

Bugungi yechim: transfer learning

- 1 Model katta **yorliqsiz** korpusda oldindan o'qitiladi: umumiy til bilimi o'rganiladi.
- 2 Keyin u kichik **yorliqlangan** datada vazifaga moslab sozlanadi (*fine-tuning*).
- 3 Natijada bilim **ko'chiriladi**: kamroq data, kamroq hisob va tezroq moslashuv.

BERT va T5 — Transformer arxitekturasida oldindan o'qitilgan modellar. Biz ularni o'z vazifamizga moslaymiz.

Prezentatsiya rejasi

- 1 Kirish: noldan o'qitmaymiz, oldindan o'qitilganni olamiz
- 2 Transfer learning: pre-training va fine-tuning
- 3 BERT: ikki tomonlama enkoder
- 4 T5: hamma narsa matndan matnga
- 5 Hugging Face ekotizimi
- 6 Xulosa va amaliyotga ko'prik

Intuitsiya: umumiy bilim $\rightarrow$ vazifaga moslash

Model ham avval umumiy til naqshlarini o'rganadi:

- **Pre-training** (*oldindan o'qitish*): katta **yorliqsiz** matn korpusida umumiy til bilimi o'rganiladi.
- **Fine-tuning** (*vazifaga moslab sozlash*): nisbatan kichik **yorliqlangan** datada model aniq vazifaga moslashtiriladi.

Masalan: avval umumiy tilni tushunish, keyin sentiment, NER yoki savol-javob kabi vazifaga moslash.

Asosiy g'oya

Oldindan o'qitilgan vaznlar modelga yaxshi **boshlang'ich nuqta** beradi.

Fine-tuning bu vaznlarni noldan yaratmaydi; ularni **vazifaga mos ravishda yangilaydi**.

umumiy model $\implies$ vazifa modeli

Pre-training va fine-tuning: ikki bosqich

Jihat	Pre-training	Fine-tuning
Ma'lumot	katta yorliqsiz matn korpusi	kichikroq yorliqlangan ma'lumotlar to'plami
Maqsad	umumiy til naqshlarini o'rganish	aniq NLP vazifasiga moslash
O'qitish signali	o'z-o'zini nazorat (<i>self-supervised</i>)	nazorat ostida (<i>supervised</i>)
Hisoblash narxi	juda katta: ko'p GPU-soat	nisbatan kichik: bir necha epoch
Natija	BERT/T5 kabi umumiy model	vazifaga moslashtirilgan model

Asosiy g'oya

Transfer learningning amaliy mazmuni: **bir marta** qimmat pre-training va **ko'p vazifalarda** nisbatan arzon fine-tuning. Amaliyotda biz odatda tayyor *pre-trained* modeldan boshlaymiz.

Fine-tuning yo'qotishi: sigmoid va BCE

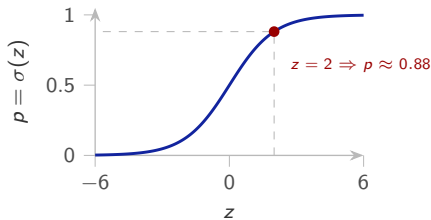
Ta'rif: binary task head va BCE yo'qotish

Model matnni ichki **vektorli ifodaga** aylantiradi: $h_i = h_\theta(x_i)$. Binary task head shu vektordan logit chiqaradi:

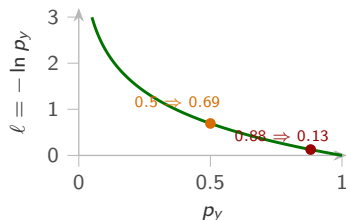
$$z_i = Wh_i + b, \quad p_i = \sigma(z_i) = \frac{1}{1 + e^{-z_i}}, \quad y_i \in \{0, 1\}.$$

$$p_{y_i} = y_i p_i + (1 - y_i)(1 - p_i), \quad \ell_i = -\ln p_{y_i}, \quad \mathcal{L}_{\text{FT}} = \frac{1}{m} \sum_{i=1}^m \ell_i.$$

1) Sigmoid: logit $\rightarrow$ ehtimol



2) BCE: ehtimol $\rightarrow$ yo'qotish



Izoh: h_i — matnning model ichidagi vektorli ifodasi; z_i — task head chiqargan logit; p_i — ijobiy sinf ehtimoli. p_{y_i} haqiqiy sinfga berilgan ehtimol bo'lgani uchun u katta bo'lsa, BCE yo'qotish kichik bo'ladi.

Nega avval pre-training, keyin fine-tuning?

1. Noldan boshlash qimmat

Tasodifiy boshlang'ich vaznlar kuchli model uchun katta data va ko'p hisoblash resursini talab qiladi.

3. Vaznlar yaxshi boshlang'ich nuqta

O'rganilgan bilim model parametrlarida saqlanadi. Shu sababli fine-tuning noldan boshlamaydi.

2. Avval umumiy til bilimi

Pre-training katta **yorliqsiz** korpusda bajariladi: model so'z, sintaksis va kontekst naqshlarini o'rganadi.

4. Keyin vazifaga moslash

Kichik **yorliqlangan** datada model vazifaga moslanadi. Ikkilik klassifikatsiya misolida BCE yo'qotishi minimallashtiriladi.

Xulosa

Pre-training modelga umumiy til bilimni beradi; fine-tuning esa shu bilimni aniq vazifaga moslashtiradi. Keyingi qadamda BCE ni qo'lda hisoblaymiz.

Qo'lda hisob: logit $z = 2$ uchun BCE

Ijobiy sharh: logit $z = 2.0$, yorliq $y = 1$

Model ijobiy sinf uchun logit chiqaradi. Avval logit ehtimolga aylantiriladi: $e^{-2.0} \approx 0.1353$.

Sigmoid va BCE:

$$p = \sigma(2.0) = \frac{1}{1 + e^{-2.0}} \approx \frac{1}{1 + 0.1353} \approx \mathbf{0.8808},$$

$$\text{BCE} = -\ln(p) = -\ln(0.8808) \approx \mathbf{0.1269} \approx \mathbf{0.127}.$$

Agar o'quv misolida yaxlitlasak: $\text{BCE} \approx 0.128$.

Model ijobiy sinfga taxminan 0.881 ehtimol berdi. Yorliq ham $y = 1$, shuning uchun yo'qotish **kichik**: taxminan 0.127.

P13 dagi test uchun yaxlitlangan qiymat:

```
assert abs(loss - 0.128) < 1e-2
```

Ma'ruza L13 [I1]-slayd

Sizning vazifangiz: nol logit uchun BCE

Savol

Model hali aniq qaror qilmagan: logit $z = 0.0$, haqiqiy yorliq $y = 1$ (ijobiy sinf).

1-qadam: $p = \sigma(0)$ nechaga teng?

2-qadam: $y = 1$ bo'lsa,

$$\text{BCE} = -\ln(p) = ?$$

Eslatma: $\sigma(0) = 0.5$, $\ln 2 \approx 0.693$.

Sizning vazifangiz: nol logit uchun BCE

Savol

Model hali aniq qaror qilmagan: logit $z = 0.0$, haqiqiy yorliq $y = 1$ (ijobiy sinf).

1-qadam: $p = \sigma(0)$ nechaga teng?

2-qadam: $y = 1$ bo'lsa,

$$\text{BCE} = -\ln(p) = ?$$

Eslatma: $\sigma(0) = 0.5$, $\ln 2 \approx 0.693$.

Javob

$$p = \sigma(0) = 0.5, \quad \text{BCE} = -\ln(0.5) = \ln 2 \approx \mathbf{0.693}.$$

Demak, 50/50 ehtimolli bashoratning yo'qotishi 0.693. Oldingi ishonchliroq to'g'ri bashoratda esa yo'qotish taxminan 0.127 edi. Fine-tuningning maqsadi — bunday yo'qotishni kamaytirish.

Kod ↔ formula: BCEWithLogitsLoss

```
import torch
import torch.nn as nn

logit = torch.tensor([2.0])    # model chiqishi: z
target = torch.tensor([1.0])   # haqiqiy yorliq: y=1

loss_fn = nn.BCEWithLogitsLoss()
loss = loss_fn(logit, target)   # sigmoid + BCE

p = torch.sigmoid(logit)
print(p.item())                # 0.8808
print(loss.item())             # 0.1269
```

Kod → formula:

Kod	Ma'nosi
logit	z
target	$y \in \{0, 1\}$
sigmoid	$p = \sigma(z)$
loss	$\ell(z, y)$

Muhim eslatma

BCEWithLogitsLoss logitni oladi va sigmoidni ichida barqaror hisoblaydi:

$$\ell = -[y \ln \sigma(z) + (1 - y) \ln(1 - \sigma(z))].$$

Bu misolda $y = 1$, shuning uchun $\ell = -\ln \sigma(2) \approx 0.1269$.

Fine-tuning tuzog'i: model oldingi bilimni unutishi mumkin

Catastrophic forgetting

Fine-tuning juda agressiv bajarilsa, model oldindan o'rgangan umumiy til bilimining bir qismini yo'qotishi mumkin.

Bunga ko'pincha quyidagilar sabab bo'ladi:

- katta learning rate η ;
- kichik datada juda ko'p epoch;
- vazifaga haddan tashqari moslashish.

Amaliy ehtiyot choralar

Fine-tuningni ehtiyotkor bajarish kerak:

- kichik learning rate: masalan, 2×10^{-5} ;
- kam epoch: odatda 2–3;
- warmup: η ni boshida asta oshirish;
- validation loss orqali overfittingni kuzatish.

Maqsad: modelni yangi vazifaga moslash, lekin oldindan o'rgangan umumiy til bilimini keskin buzib yubormaslik.

Zamonaviy fine-tuning: PEFT va LoRA

To'liq fine-tuning

Klassik fine-tuningda modelning ko'p yoki barcha vaznlari yangilanadi:

$$\theta \leftarrow \theta - \eta \nabla_{\theta} \mathcal{L}.$$

Bu kuchli, lekin:

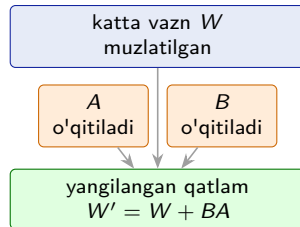
- GPU/xotira talabi yuqori;
- kichik datada overfitting xavfi bor;
- oldingi bilim qisman buzilishi mumkin.

PEFT g'oyasi

PEFT (*Parameter-Efficient Fine-Tuning*) — katta model vaznlarining aksar qismini muzlatib, faqat kichik qo'shimcha parametrlarni o'qitish.

Amaliy eslatma

Bu kursda asosiy amaliyot **Trainer** bilan odatiy fine-tuning oqimida beriladi. Real loyihalarda esa xotira va GPU resursini tejash uchun PEFT/LoRA yondashuvlari ko'p ishlatiladi.



LoRA: katta W emas, kichik A , B o'qitiladi

LoRA nima qiladi?

LoRA (*Low-Rank Adaptation*) katta vazn W ni o'zgartirmaydi. Unga kichik o'qitiladigan qo'shimcha BA qo'shadi:

$$W' = W + BA.$$

Prezentatsiya rejasi

- 1 Kirish: noldan o'qitmaymiz, oldindan o'qitilganni olamiz
- 2 Transfer learning: pre-training va fine-tuning
- 3 BERT: ikki tomonlama enkoder**
- 4 T5: hamma narsa matndan matnga
- 5 Hugging Face ekotizimi
- 6 Xulosa va amaliyotga ko'prik

Intuitsiya: BERT so'zni ikkala tomondan tushunadi

BERT nima?

BERT(Bidirectional Encoder Representations from Transformers) — Transformer **enkoder** stegi. Unda har bir token self-attention orqali gapdagi boshqa tokenlar bilan bog'lanadi.

Ikki tomonlama kontekst:

BERT so'z ma'nosini faqat chapdan emas, balki **chap va o'ng** kontekstdan birga aniqlaydi.

Misol

yuz so'zi kontekstga qarab turlicha ma'no beradi:
yuz so'm → son miqdori; *yuz ifodasi* → chehra.

Fine-tuningda qanday ishlatiladi?

Klassifikatsiyada [CLS] tokenining oxirgi vektori butun gap vakili sifatida olinadi va unga task head qo'shiladi.

NER kabi token-level vazifalarda esa har bir token vektoriga alohida klassifikator qo'llanadi.

Muhim cheklov

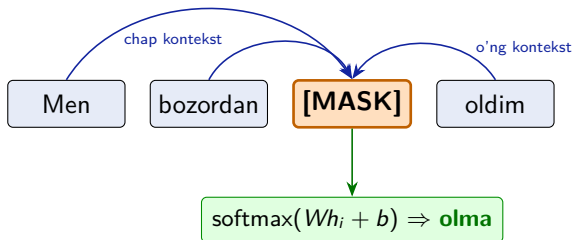
BERT dekodersiz enkoder modelidir. Shuning uchun u GPT kabi chapdan o'ngga matn generatsiyasi uchun emas, ko'proq **tushunish** vazifalari uchun ishlatiladi.

BERT pre-training: Masked Language Modeling (MLM)

Ta'rif: MLM (*niqoblangan til modellashtirish*)

Matndan ayrim tokenlar niqoblanadi va model ularni qolgan kontekstdan tiklashni o'rganadi:

$$\mathcal{L}_{\text{MLM}} = - \sum_{i \in M} \ln p(x_i | \tilde{x}), \quad p(x_i | \tilde{x}) = \text{softmax}(Wh_i + b).$$



Izoh: M — niqoblangan pozitsiyalar to'plami ($\approx 15\%$); $\tilde{x}$ — niqoblangan kirish; h_i — i -pozitsiyadagi enkoder vektori. softmax lug'at bo'yicha ehtimollarni beradi; model niqoblangan tokenni **chap va o'ng kontekstdan** tiklaydi.

Nega BERTda niqoblash kerak?

1. Chapdan bashorat cheklangan

Avtoregressiv til modeli keyingi so'zni faqat **chap kontekst** asosida bashorat qiladi. O'ng tomondagi so'zlar ishlatilmaydi.

2. Ikki tomonlamalikda muammo bor

Agar target so'z yashirilmasa, self-attention orqali model bashorat qilinayotgan so'zning **o'zini ko'rib qo'yadi**. Bu vazifani juda oson va ma'nosiz qiladi.

3. Yechim: [MASK]

Ayrim tokenlar [MASK] bilan almashtiriladi. Model yashirilgan tokenni qolgan kontekstga qarab topishi kerak.

4. Natija

Endi model **chap va o'ng kontekstdan** foydalanadi, lekin javobni oldindan ko'rib qo'ymaydi. Shu orqali ikki tomonlama til bilimi o'rganiladi.

Xulosa

MLM niqoblash orqali ikki narsani birlashtiradi: **ikki tomonlama kontekst** va **javobni yashirib bashorat qilish**. Keyingi slaydda bitta niqoblangan so'zni qo'lda bashorat qilamiz.

Qo'lda misol: [MASK] tokenini bashorat qilish

Soddalashtirilgan lug'at: $\{kitob, olma, non\}$

Gap: "Men bozordan [MASK] oldim." Model logitlari:

$$s_{kitob} = 1, \quad s_{olma} = 3, \quad s_{non} = 2.$$

Softmax hisoblash:

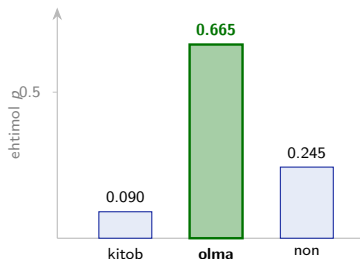
$$Z = e^1 + e^3 + e^2 = 2.718 + 20.086 + 7.389 = 30.193,$$

$$p_j = \frac{e^{s_j}}{Z}.$$

so'z	$p(\text{so'z})$
kitob	$2.718/30.193 \approx 0.090$
olma	$20.086/30.193 \approx \mathbf{0.665}$
non	$7.389/30.193 \approx 0.245$

Eng katta ehtimol **olma** uchun chiqdi. Demak, model [MASK] o'rniga "olma" tokenini bashorat qiladi.

Softmax ehtimollari



Logitlar: $[1, 3, 2] \Rightarrow$ eng baland ustun = bashorat

Real BERTda lug'at minglab tokenlardan iborat bo'ladi. Bu yerda faqat 3 ta nomzod o'quv misoli sifatida olindi.

Savol

Logitlar berilgan:

$$s_a = 1, \quad s_b = 1, \quad s_c = 2.$$

$$p = ?$$

Eslatma: $e^1 \approx 2.718$, $e^2 \approx 7.389$. Qaysi sinf eng katta ehtimol oladi?

Sizning vazifangiz: softmax bilan bashorat

Savol

Logitlar berilgan:

$$s_a = 1, \quad s_b = 1, \quad s_c = 2.$$

$$p = ?$$

Eslatma: $e^1 \approx 2.718$, $e^2 \approx 7.389$. Qaysi sinf eng katta ehtimol oladi?

Javob

$$Z = e^1 + e^1 + e^2 = 2.718 + 2.718 + 7.389 = 12.825.$$

sinf	$p(\text{sinf})$
a	$2.718/12.825 \approx 0.212$
b	$2.718/12.825 \approx 0.212$
c	$7.389/12.825 \approx \mathbf{0.576}$

Demak, c eng ehtimolli sinf. a va b logitlari teng bo'lgani uchun ularning ehtimollari ham teng chiqdi.

Kod ↔ formula: BERT [CLS] vektori

```
from transformers import AutoTokenizer, AutoModel

name = "bert-base-multilingual-cased"
tok = AutoTokenizer.from_pretrained(name)
model = AutoModel.from_pretrained(name)

inp = tok("Bu film juda zo'r", return_tensors="pt")
out = model(**inp)

cls = out.last_hidden_state[:, 0, :] # [CLS] vektori
print(cls.shape)                    # torch.Size([1, 768])
```

Kod → ma'no:

Kod	Ma'nosi
from_pretrained	oldindan o'qitilgan model
last_hidden_state	token vektorlari
[:, 0, :]	[CLS] vektori h_{CLS}

Fine-tuning g'oyasi

Klassifikatsiyada [CLS] vektori gap vakili sifatida olinadi:

$$z = Wh_{CLS} + b, \quad p = \sigma(z).$$

So'ng task head BCE yoki boshqa loss orqali fine-tune qilinadi.

Eslatma: bu kod [CLS] vektorini olishni ko'rsatadi. Amaliy klassifikatsiyada task head qo'shiladi yoki AutoModelForSequenceClassification ishlatiladi.

BERT cheklovi: GPT kabi ketma-ket matn yaratmaydi

Nima uchun?

BERT — **enkoder-only** model. Unda GPT dagidek autoregressiv dekoder yo'q.

Shuning uchun u matnni chapdan o'ngga **tokenma-token yaratish** uchun asosiy tanlov emas.

Muhim farq

BERT [MASK] o'rnidagi tokenni bashorat qiladi. Bu — niqoblangan tokenni tiklash; GPT kabi erkin matn davom ettirish emas.

Qaysi vazifaga qaysi model?

Vazifa	Qulay model
Klassifikatsiya, NER	BERT tipidagi enkoder
Tarjima, xulosa	T5/BART tipidagi seq2seq
Matn davom ettirish	GPT tipidagi dekoder

Ko'prik: BERT tushunish vazifalarida kuchli. Keyingi bo'limda T5 ni ko'ramiz: u vazifalarni **matndan matnga** formatida yechadi.

Prezentatsiya rejasi

- 1 Kirish: noldan o'qitmaymiz, oldindan o'qitilganni olamiz
- 2 Transfer learning: pre-training va fine-tuning
- 3 BERT: ikki tomonlama enkoder
- 4 T5: hamma narsa matndan matnga**
- 5 Hugging Face ekotizimi
- 6 Xulosa va amaliyotga ko'prik

T5 intuitsiyasi: hamma vazifa matndan matnga

T5 nima?

T5 (*Text-to-Text Transfer Transformer*) — Transformer **enkoder-dekoder** modelidir.

Asosiy g'oya:

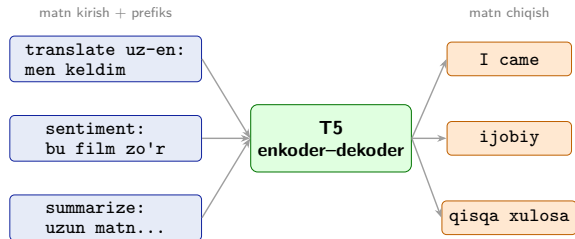
Har bir NLP vazifasi bitta umumiy formatga keltiriladi:

matn kirish → matn chiqish.

Masalan, tarjima, xulosa va klassifikatsiya ham matn chiqarish sifatida qaraladi.

Muhim farq

Vazifa **prefiks** orqali beriladi: `translate:`, `summarize:`, `sentiment:`. Model javobni ham matn ko'rinishida generatsiya qiladi.



Klassifikatsiyada ham model raqamli sinf emas, **label matni** chiqaradi: masalan, *ijobiy* yoki *salbiy*.

T5: matndan matnga yo'qotish

Ta'rif: ketma-ketlik kross-entropiyasi

$$\mathcal{L} = - \sum_{t=1}^T \ln p(y_t \mid y_{<t}, x)$$

Bu yerda x — **prefiksli kirish matni**, y_t — chiqishning t -tokeni, $y_{<t}$ — undan oldingi chiqish tokenlari.

Qo'lda misol

Kirish: translate uz-en: men keldim

Maqsadli chiqish: i came

Demak yo'qotish ikki had yig'indisidan iborat:

$$\mathcal{L} = - \ln p(i \mid x) - \ln p(\text{came} \mid i, x).$$

Asosiy g'oya

T5 chiqishni **tokenma-token** generatsiya qiladi. Har qadamda dekoder oldingi chiqish tokenlari va enkoder bergan kirishga tayan-gan holda keyingi tokenni bashorat qiladi.

Pre-training eslatmasi

T5 pre-trainingda *span corruption* ish-latadi: matnning ayrim bo'laklari yashiriladi, dekoder esa ularni matn sifatida tiklaydi.

Nega T5 yagona matndan-matnga format ishlatadi?

1. Klassik yondashuvda muammo

Klassifikatsiya, tarjima, xulosa yoki savol-javob vazifalarida chiqish ko'rinishi turlicha bo'ladi. Shuning uchun ko'pincha alohida task head yoki alohida o'qitish sxemasi kerak bo'ladi.

2. T5 g'oyasi

Agar hamma vazifada chiqishni **matn** deb qarasak, barcha vazifalar bir xil ko'rinishga keladi:

prefiksli kirish → matn chiqish.

3. Natija

Vazifa prefiks bilan beriladi: `translate:`, `summarize:`, `sentiment:`. Model esa chiqishni tokenma-token matn sifatida hosil qiladi.

4. O'qitish

Asosiy loss — ketma-ketlik kross-entropiyasi: model har qadamda keyingi chiqish tokenini bashorat qiladi.

Xulosa

Matndan-matnga format T5 uchun yagona interfeys beradi: vazifa prefiks orqali aytiladi, javob esa matn sifatida chiqariladi. Keyingi slaydda bitta chiqish tokeni uchun loss hisoblaymiz.

Qo'lda misol: T5 chiqish tokeni yo'qotishi

Sentiment vazifasi matndan-matnga formatda

Kiritish: "sentiment: bu film zo'r". Kutilgan chiqish: "ijobiy".

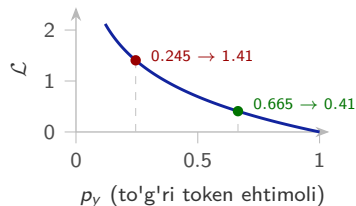
Soddalashtirilgan o'quv misolida "ijobiy" bitta chiqish tokeni deb olinadi. Model shu token uchun $p(\text{ijobiy}) = 0.665$ ehtimol berdi.

Bir token uchun kross-entropiya:

$$\begin{aligned}\mathcal{L} &= -\ln p(y_1 | x) \\ &= -\ln p(\text{ijobiy} | x) \\ &= -\ln(0.665) \approx \mathbf{0.408}.\end{aligned}$$

T5 da javob tokenlarga ajraladi. Har bir chiqish tokeni uchun softmax kross-entropiya hisoblanadi; umumiy loss tokenlar bo'yicha yig'iladi.

Token loss: $\mathcal{L} = -\ln p_y$



To'g'ri token ehtimoli oshsa, yo'qotish kamayadi.

Sizning vazifangiz: past ehtimol nimani anglatadi?

Savol

Oldingi misolda to'g'ri token ehtimoli $p_y = 0.665$ bo'lganda $\mathcal{L} \approx 0.408$ edi.

Endi model to'g'ri tokenga faqat

$$p_y = 0.245$$

ehtimol berdi.

1-savol: yo'qotish 0.408 dan katta bo'ladimi yoki kichik?

2-savol: $\mathcal{L} = -\ln(0.245)$ taxminan nechaga teng?

Sizning vazifangiz: past ehtimol nimani anglatadi?

Savol

Oldingi misolda to'g'ri token ehtimoli $p_y = 0.665$ bo'lganda $\mathcal{L} \approx 0.408$ edi.

Endi model to'g'ri tokenga faqat

$$p_y = 0.245$$

ehtimol berdi.

1-savol: yo'qotish 0.408 dan katta bo'ladimi yoki kichik?

2-savol: $\mathcal{L} = -\ln(0.245)$ taxminan nechaga teng?

Javob

$$\mathcal{L} = -\ln(0.245) \approx \mathbf{1.406}.$$

to'g'ri token ehtimoli	yo'qotish
0.665	0.408
0.245	1.406

Demak, model to'g'ri token ehtimolini past bergan sari yo'qotish kattalashadi. Fine-tuning shu yo'qotishni kamaytirishga va to'g'ri token ehtimolini oshirishga intiladi.

Kod ↔ formula: T5 matndan matnga

```
from transformers import AutoTokenizer
from transformers import AutoModelForSeq2SeqLM

name = "t5-small"    # inglizcha T5 modeli
tok = AutoTokenizer.from_pretrained(name)
model = AutoModelForSeq2SeqLM.from_pretrained(name)

text = "summarize: NLP models learn patterns from text."
inp = tok(text, return_tensors="pt")

ids = model.generate(**inp, max_new_tokens=20)
print(tok.decode(ids[0], skip_special_tokens=True))
```

Eslatma: bu kod inference, ya'ni matn generatsiyasini ko'rsatadi. Fine-tuningda esa maqsadli chiqish labels sifatida beriladi va ketma-ketlik kross-entropiyasi minimallashtiriladi.

Kod → ma'no:

Kod	Ma'nosi
summarize:	vazifa prefiksi
AutoModelForSeq2SeqLM	enkoder-dekoder LM
generate	tokenma-token dekoding

Formula bilan bog'lanishi

Prefiks vazifani bildiradi. Dekoder har qadamda keyingi token ehtimolini hisoblaydi:

$$p_{\theta}(y_t \mid y_{<t}, x).$$

generate shu ehtimollar asosida chiqish matnini hosil qiladi.

T5 tuzog'i: prefiks noto'g'ri bo'lsa, vazifa ham chalkashadi

Muammo: prefiksni unutish yoki almashtirish

T5 kirishdagi **prefiks** orqali vazifani aniqlaydi. Prefiks yo'q yoki noto'g'ri bo'lsa, model vazifani chalkashtirishi mumkin.

Masalan, `summarize:` o'rniga `translate:` yozilsa, model xulosa emas, tarjima qilishga urinadi.

Yechim

Har vazifa uchun **aniq va izchil prefiks** ishlatish. Prefiks tili va shakli model qanday o'qitilganiga mos bo'lishi kerak.

prefiks vazifani boshqaradi

Noto'g'ri: `translate:` uzun matn...

kutilmagan chiqish: tarjima

To'g'ri: `summarize:` uzun matn...

kutilgan chiqish: qisqa xulosa

Amaliy qoida: fine-tuning va inference vaqtida prefikslar bir xil bo'lsin. Aks holda treningdagi vazifa formati bilan testdagi format mos kelmaydi.

BERT va T5: qaysi birini tanlash?

Jihat	BERT	T5
Arxitektura	faqat enkoder	enkoder–dekoder
Kirish/chiqish	matn → vektor, yorliq yoki token yorlig'i	matn → matn
Pre-training	MLM (<i>Masked Language Modeling</i>)	span corruption (<i>text infilling</i>)
Generativ ishlatish	erkin matn generatsiyasi uchun asosiy tanlov emas	generativ vazifalar uchun qulay
Qulay vazifalar	klassifikatsiya, NER, sentiment, juft matnni taqqoslash	xulosa, tarjima, savol-javob, matn yaratish

Amaliy qoida

Agar vazifa asosan **tushunish va yorliqlash** bo'lsa, ko'pincha **BERT** qulayroq. Agar vazifa **matndan matnga generatsiya** bo'lsa, odatda **T5** qulayroq. Lekin yakuniy tanlov **dev/validation natijasi**, tezlik va resurs talabi bilan tasdiqlanadi.

Prezentatsiya rejasi

- 1 Kirish: noldan o'qitmaymiz, oldindan o'qitilganni olamiz
- 2 Transfer learning: pre-training va fine-tuning
- 3 BERT: ikki tomonlama enkoder
- 4 T5: hamma narsa matndan matnga
- 5 Hugging Face ekotizimi**
- 6 Xulosa va amaliyotga ko'prik

Hugging Face: tayyor model va kutubxonalar ekotizimi

Model Hub

Oldindan o'qitilgan modellar ombori: BERT, T5, RoBERTa, XLM-R va boshqa checkpoint-lar.

Kutubxonalar

transformers, datasets, tokenizers, evaluate kabi vositalar model bilan ishlashni soddalashtiradi.

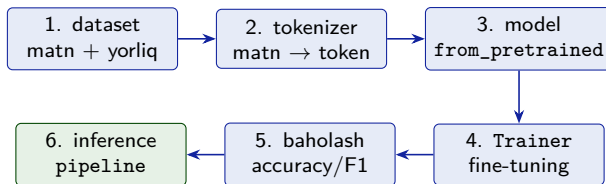
Amaliy foyda

Modelni noldan yozmaymiz: tayyor modelni yuklaymiz, datasetimizga moslaymiz va baholaymiz.

Asosiy g'oya

Hugging Face — faqat bitta kutubxona emas, balki **tayyor modellar + tokenizerlar + datasetlar + trening/baholash vositalari** ekotizimidir. Keyingi slaydda shu ekotizim bilan fine-tuning oqimini ko'ramiz.

Hugging Face fine-tuning oqimi



Bu slayd amaliy ketma-ketlikni ko'rsatadi: avval dataset tayyorlanadi, keyin tokenizatsiya qilinadi, tayyor model yuklanadi, Trainer orqali fine-tune qilinadi, validation/testda baholanadi va yakunda inference uchun ishlatiladi.

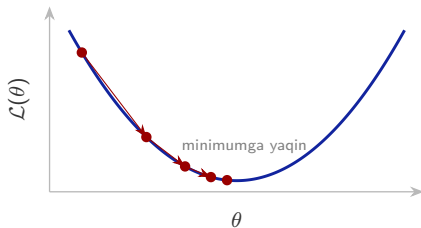
Fine-tuning yangilanishi: soddalashtirilgan rasmiy ko'rinish

Ta'rif: parametr yangilanishi va linear warmup

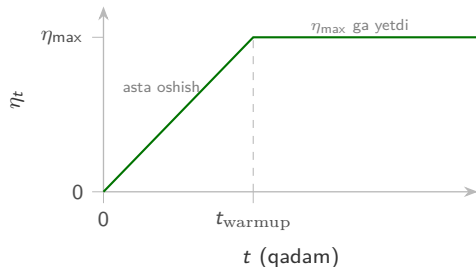
Fine-tuningda parametrlar yo'qotishni kamaytirish tomonga yangilanadi (*soddalashtirilgan ko'rinish*):

$$\theta \leftarrow \theta - \eta_t \nabla_{\theta} \mathcal{L}, \quad \eta_t = \eta_{\max} \cdot \min\left(1, \frac{t}{t_{\text{warmup}}}\right).$$

Intuitsiya: lossni kamaytirish



Warmup jadvali



Izoh: θ — model parametrlari, $\mathcal{L}$ — fine-tuning yo'qotishi, η_t — t -qadamdagi o'rganish tezligi. Warmup boshida η_t ni kichikdan boshlab asta oshiradi; bu oldindan o'qitilgan vaznlarni keskin buzib yubormaslikka yordam beradi. Amaliyotda optimizator ko'pincha AdamW, scheduler esa warmupdan keyin pasayuvchi ham bo'lishi mumkin.

Nega Hugging Face ekotizimi qulay?

Muammo: tarqoq modellar

Modellar turli nom, turli format va turli API bilan keladi. Har birini alohida yuklash va ishlatish no-qulay.

Muammo: takrorlanuvchi trening kodi

Trening loopi, batchlash, logging, evaluation kabi bosqichlar ko'p vazifada qayta-qayta yoziladi.

Yechim: yagona yuklash interfeysi

`from_pretrained` ko'p model va tokenizatorni bir xil usulda yuklashga imkon beradi. Shu sabab kod bir xil qoladi, model esa almashishi mumkin.

Yechim: standart dataset va Trainer

`datasets` ma'lumot bilan ishlashni, `Trainer` esa trening/evaluation jarayonini soddalashtiradi. Natijada kod hajmi ancha qisqaradi.

Xulosa

Hugging Face ekotizimi **yuklashni standartlashtiradi**, **takroriy kodni kamaytiradi** va fine-tuningni tezroq sinab ko'rishga yordam beradi. Keyingi slaydda o'qitish qadamlari (*training steps*) sonini hisoblaymiz.

Qo'lda misol: fine-tuning qadamlar soni

Trainer konfiguratsiyasi

$N = 5000$ namuna, effective batch size $B = 16$, epoch $E = 3$. Bu misolda gradient accumulation = 1 va drop_last=False.

Hisob:

$$\text{qadam/epoch} = \left\lceil \frac{N}{B} \right\rceil = \left\lceil \frac{5000}{16} \right\rceil = \lceil 312.5 \rceil = 313.$$

$$\text{jami qadam} = 313 \times E = 313 \times 3 = \mathbf{939}.$$

$\lceil \cdot \rceil$ — oxirgi to'liq bo'lmagan batch ham alohida qadam sifatida hisoblanadi.

Xulosa

Bu konfiguratsiyada fine-tuning jami **939** training step bajaradi.

Oldindan o'qitilgan modeldan boshlaganimiz uchun bu jarayon noldan o'qitishga qaraganda odatda kamroq resurs talab qiladi.

Eslatma: multi-GPU yoki gradient accumulation ishlatilsa, B effective batch size sifatida qayta hisoblanadi.

Sizning vazifangiz: fine-tuning qadamlarini hisoblash

Savol

Dataset: $N = 8100$ namuna. Batch size: $B = 32$. Epoch: $E = 2$.

Hisoblang:

qadam/epoch =? jami qadam =?

Eslatma: agar oxirgi batch to'liq bo'lmasa ham, u alohida training step sifatida hisoblanadi.

Sizning vazifangiz: fine-tuning qadamlarini hisoblash

Savol

Dataset: $N = 8100$ namuna. Batch size: $B = 32$. Epoch: $E = 2$.

Hisoblang:

qadam/epoch = ? jami qadam = ?

Eslatma: agar oxirgi batch to'liq bo'lmasa ham, u alohida training step sifatida hisoblanadi.

Javob

$$\text{qadam/epoch} = \left\lceil \frac{8100}{32} \right\rceil = \lceil 253.125 \rceil = 254.$$

$$\text{jami qadam} = 254 \times 2 = \mathbf{508}.$$

Demak, har epochda 254 ta training step bajariladi. 2 epoch uchun jami 508 qadam bo'ladi.

Kod ↔ formula: Hugging Face Trainer

```
from transformers import (
    AutoModelForSequenceClassification,
    TrainingArguments, Trainer
)
model = AutoModelForSequenceClassification.from_pretrained(
    "distilbert-base-multilingual-cased",
    num_labels=2
)
args = TrainingArguments(
    output_dir="out",
    learning_rate=2e-5,
    per_device_train_batch_size=16,
    num_train_epochs=3,
    warmup_steps=100
)
trainer = Trainer(
    model=model,
    args=args,
    train_dataset=tokenized_ds
)
trainer.train()
```

Kod → formula:

Kod	Ma'nosi
from_pretrained	boshlang'ich vaznlar θ_0
learning_rate	η
warmup_steps	t_{warmup}
num_labels=2	$z \in \mathbb{R}^2$

Loss nima?

Bu kodda model 2 ta logit chiqaradi:

$$p = \text{softmax}(z), \quad \mathcal{L} = -\ln p_y.$$

Trainer shu lossni kamaytirish uchun model vaznlarini yangilaydi.

Hugging Face tuzog'i: model va tokenizator mos bo'lishi kerak

Muammo: nomuvofiq tokenizator

Tokenizator matnni **token ID**larga aylantiradi. Model esa shu ID larni o'z lug'ati (*vocabulary*) tartibi bo'yicha talqin qiladi.

Agar tokenizator boshqa checkpointdan olinsa, token ID $\leftrightarrow$ so'z/subtoken mosligi buzilishi mumkin. Natijada model noto'g'ri yoki ma'nosiz chiqish berishi mumkin.

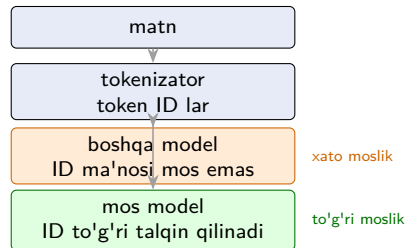
Amaliy qoida

Odatda model va tokenizator **bir xil checkpoint**dan yuklanadi:

```
AutoTokenizer.from_pretrained(name)
```

```
AutoModel.from_pretrained(name)
```

Bu ularning vocabulary va token ID tartibi mos bo'lishini ta'minlaydi.



Alohida eslatma

Klassifikatsiyada `num_labels` ham vazifaga mos bo'lishi kerak: binary sentiment uchun 2 sinf, ko'p sinfli vazifada esa sinflar soni. Bu tokenizator mosligidan **alohida** sozlama.

Prezentatsiya rejasi

- 1 Kirish: noldan o'qitmaymiz, oldindan o'qitilganni olamiz
- 2 Transfer learning: pre-training va fine-tuning
- 3 BERT: ikki tomonlama enkoder
- 4 T5: hamma narsa matndan matnga
- 5 Hugging Face ekotizimi
- 6 Xulosa va amaliyotga ko'prik

O'zbek tilida: tokenizatsiya va qo'shimchalar

1. mBERT va WordPiece

O'zbek tili **agglutinativ**: so'zga bir nechta qo'shimcha ketma-ket ulanadi. Shu sabab bitta so'z bir nechta **subtoken**ga ajralishi mumkin.

Namuna

o'rganaman → o'r ##gan ##aman

Bu faqat tushuntirish uchun namunaviy ko'rinish; haqiqiy bo'linish tokenizator lug'atiga (*vocabulary*) bog'liq.

Muammo

Subtokenlar ko'paygan sari ketma-ketlik uzayadi. Bu tezlik, xotira sarfi va token-level vazifalarda yorliq moslashiga ta'sir qilishi mumkin.

2. XLM-RoBERTa va SentencePiece

XLM-RoBERTa ko'p tilli katta korpuslarda o'qitilgan va SentencePiece tokenizatsiyasidan foydalanadi. O'zbek vazifalarida uni mBERT bilan alohida solishtirish kerak.

Amaliy tanlov

Model tanlashda faqat nomiga qaramaymiz:

- tokenlar soni qanchaga oshyapti?
- validation accuracy/F1 qanday?
- inference tezligi va xotira talabi qanday?

Qoida

O'zbek NLP vazifasida mBERT ham, XLM-RoBERTa ham nomzod bo'lishi mumkin. Yakuniy qaror tokenizator tahlili, validation natijasi va resurs talabi bilan tasdiqlanadi.

O'zbek NLP vazifasida model tanlash checklisti

Faqat model nomiga qaramaymiz

mBERT, XLM-RoBERTa, DistilBERT yoki T5 tanlashda asosiy savol: **shu vazifa va shu datasetda qaysi biri yaxshi?**

Model tanlash nazariy nom bilan emas, tajriba natijasi bilan tasdiqlanadi.

Tekshiriladigan mezonlar

- Tokenizator matnni qanchaga uzaytiradi?
- Validation accuracy/F1 qanday?
- Inference tezligi va xotira talabi qanday?
- Dataset hajmi va yorliq sifati yetarlimi?

Xulosa

Model tanlash — **empirik qaror**: yakuniy tanlov tokenizer tahlili, validation natijasi, tezlik/xotira talabi va vazifa turiga qarab tasdiqlanadi.

Vazifa turi

Boshlang'ich nomzod

Sentiment / klassifikatsiya

mBERT, XLM-RoBERTa, DistilBERT

NER / token yorliqlash

mBERT, XLM-RoBERTa

Tarjima / xulosa / generatsiya

T5 yoki boshqa seq2seq model

Resurs cheklangan holat

kichikroq model yoki PEFT/LoRA

Ehtiyot bo'ling

O'zbek tilida ko'p qo'shimchali so'zlar subtokenlarga ajralishi mumkin. Shu sabab tokenlar soni, tezlik va validation metrikalari alohida tekshiriladi.

Sintez: noldan o'qitish, fine-tuning, BERT va T5

Jihat	Noldan o'qitish	Fine-tuning (transfer)
Ma'lumot	katta korpus, ko'pincha yorliqsiz	vazifaga mos kichikroq dataset, ko'pincha yorliqlangan
Hisob	juda katta: ko'p vaqt, ko'p GPU	ancha kamroq: odatda bitta yoki bir necha GPU bilan
O'zbek uchun	odatda amaliy emas	amaliy va qulay
Qulay model	noldan tanlash va o'qitish qimmat	BERT — tushunish; T5 — matn yaratish

Asosiy g'oya

Transfer learning katta korpusda o'rganilgan bilimni kichik vazifaga ko'chiradi. Agar vazifa **klassifikatsiya**, **NER**, **sentiment** bo'lsa, ko'pincha **BERT** qulay; agar **tarjima**, **xulosa**, **savol-javob** kabi matn yaratish kerak bo'lsa, **T5** mos keladi. Hugging Face esa bu jarayonni kod jihatdan ancha soddalashtiradi.

Asosiy manba: Devlin va boshqalar (2019)

Manba

Devlin, J., Chang, M.-W., Lee, K., Toutanova, K. (2019). **BERT: Pre-training of Deep Bidirectional Transformers for Language Understanding**. *NAACL-HLT 2019*.

Ahamiyati:

- Transformer **enkoderi** asosida chuqur ikki tomonlama kontekstli til modelini ko'rsatdi.
- Masked Language Modeling (MLM) orqali **pre-training** va keyingi **fine-tuning** paradigmasini ommalashtirdi.
- Klassifikatsiya, NER va savol-javob kabi tushunish vazifalarida oldindan o'qitilgan modeldan foydalanish samaradorligini ko'rsatdi.

Muhokama savoli

Agar mBERT tokenizeri o'zbekcha qo'shimchali so'zlarni ko'p subtokenlarga ajratsa, bu sentiment klassifikatsiyasida qanday muammo tug'dirishi mumkin? XLM-RoBERTa bilan solishtirishda qaysi metrikalarga qarash kerak?

Bugungi maqsadlar bajarildi

- 1 ✓ **Farqladik:** transfer learning, pre-training va fine-tuning bosqichlari qanday bog'lanishini.
- 2 ✓ **Tushuntirdik:** oldindan o'qitilgan modellar resurslari cheklangan tillar uchun nega amaliy jihatdan muhimligini.
- 3 ✓ **Hisobladik:** fine-tuning loss uchun oddiy BCE misoli: $\sigma(2.0) \approx 0.881$, $\text{BCE} \approx 0.127$ (≈ 0.128 ga yaxlitlanadi).
- 4 ✓ **Taqqosladik:** BERT — ikki tomonlama enkoder va MLM; T5 — matndan-matnga enkoder–dekoder yondashuvi.
- 5 ✓ **Ko'rsatdik:** Hugging Face ekotizimida model, tokenizer, dataset, Trainer va baholash orqali fine-tuning oqimini.

Keyingi ma'ruza (L14): fine-tuningdan RAGga

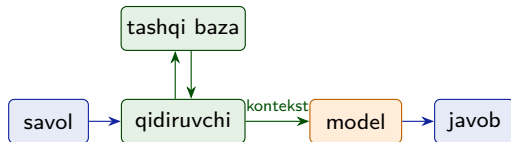
Bugungi chegarani ko'rdik:

Fine-tuning model parametrlarini vazifaga moslaydi. Lekin tez-tez yangilanadigan faktlarni faqat vaznlarga singdirish har doim qulay emas.

- Yangi fakt chiqsa, modelni qayta sozlash kerak bo'lishi mumkin.
- Parametrik bilimda manba aniq ko'rinmaydi.
- Eski faktlar model javobida saqlanib qolishi mumkin.

L14 g'oyasi

RAG (*Retrieval-Augmented Generation*) — model javob yozishdan oldin tashqi bilim bazasidan kerakli kontekstni qidiradi. Yangi bilimni bazaga qo'shish ko'pincha qayta fine-tune qilishdan yengilroq.

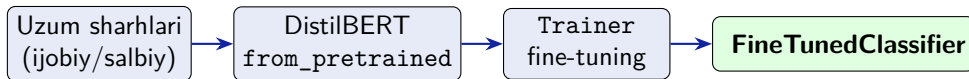


RAG: qidirish + generatsiya

Fine-tuning: bilim ko'proq model parametrlarida.

RAG: javob uchun kerakli bilim tashqi bazadan olinadi.

Amaliy dars: DistilBERTni o'zbek sentimentiga fine-tune qilish



P13 da qilasiz:

- Uzum sharhlarini sentiment yorliqlari bilan tayyorlash
- `from_pretrained` bilan DistilBERT yuklash
- Trainer orqali fine-tuning va baholash

Muhim eslatma

L13 dagi $\sigma(2.0) \approx 0.881$ va $BCE \approx 0.127$ misoli — **loss intuitsiyasi** uchun warm-up edi.

Amalda P13 da siz oldindan o'qitilgan **DistilBERT + klassifikatsiya head** ni sentiment vazifasiga moslaysiz va natijani accuracy/F1 bilan baholaysiz.

- ① Devlin, J., Chang, M.-W., Lee, K., Toutanova, K. (2019). *BERT: Pre-training of Deep Bidirectional Transformers for Language Understanding*. NAACL-HLT 2019.
- ② Raffel, C., Shazeer, N., Roberts, A., Lee, K., Narang, S., Matena, M., Zhou, Y., Li, W., Liu, P. J. (2020). *Exploring the Limits of Transfer Learning with a Unified Text-to-Text Transformer*. JMLR 21.
- ③ Howard, J., Ruder, S. (2018). *Universal Language Model Fine-tuning for Text Classification*. ACL 2018.
- ④ Conneau, A., Khandelwal, K., Goyal, N., Chaudhary, V., Wenzek, G., Guzm'an, F., Grave, E., Ott, M., Zettlemoyer, L., Stoyanov, V. (2020). *Unsupervised Cross-lingual Representation Learning at Scale*. ACL 2020.
- ⑤ Hugging Face. *Transformers documentation*: huggingface.co/docs/transformers.

Ilova S1: MLM niqoblash strategiyasi (80/10/10)

BERT pre-trainingda nima qiladi?

Matndagi tokenlarning taxminan 15% qismi tanlanadi. Bu tanlangan tokenlar quyidagicha qayta ishlanadi:

- 80% — [MASK] bilan almashtiriladi.
- 10% — tasodifiy boshqa token bilan almashtiriladi.
- 10% — o'zgarishsiz qoldiriladi.

Muhim

Loss faqat tanlangan 15% pozitsiyalar uchun hisoblanadi. Har uch holatda ham model **asl tokenni** bashorat qiladi.

100 tokenli intuitiv misol

Agar matnda 100 ta token bo'lsa:

15% $\Rightarrow$ 15 ta token tanlanadi.

Qayta ishlash	Taxminiy son
[MASK]	12
tasodifiy token	1–2
o'zgarishsiz	1–2

Nega kerak?

Fine-tuningda odatda [MASK] ko'rinmaydi. 80/10/10 strategiya pre-training va fine-tuning orasidagi nomuvofiqlikni kamaytirishga yordam beradi.

Ilova S2: T5 span-corruption pre-training

Asosiy g'oya

T5 alohida tokenni emas, **ketma-ket bo'lakni** (*span*) niqoblaydi. Niqoblangan bo'laklar maxsus **sentinel tokenlar** bilan almashtiriladi.

Soddalashtirilgan misol:

Kirish

NLP <extra_id_0> qiziq <extra_id_1> fan

Maqsad chiqish

<extra_id_0> juda <extra_id_1> foydali

Ya'ni model niqoblangan bo'laklarni **birin-ketin matn sifatida** tiklaydi.

Nima uchun qulay?

- Enkoder kirishni o'qiydi.
- Dekoder yo'qolgan bo'laklarni yozadi.
- Pre-training ham, fine-tuning ham **matn** → **matn** formatida qoladi.

Eslatma: bu yerda <extra_id_0>, <extra_id_1> — sentinel tokenlar.